

**Министерство науки и высшего образования Российской Федерации**  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ**  
**УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**  
**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**  
**ITMO University**

**ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 6**

**По дисциплине** Web-программирование

**Тема работы** Работа с сокетами

**Обучающийся** Алексеев Тимофей Юрьевич

**Факультет** Факультет инфокоммуникационных технологий

**Группа** K3321

**Направление подготовки** 11.03.02 Инфокоммуникационные технологии и  
системы связи

**Образовательная программа** Программирование в  
инфокоммуникационных системах

|                    |                             |                                          |                                  |
|--------------------|-----------------------------|------------------------------------------|----------------------------------|
| <b>Обучающийся</b> | <u>20.04.2025</u><br>(дата) | <u>                    </u><br>(подпись) | <u>Алексеев Т.Ю.</u><br>(Ф.И.О.) |
|--------------------|-----------------------------|------------------------------------------|----------------------------------|

|                     |                                       |                                          |                                  |
|---------------------|---------------------------------------|------------------------------------------|----------------------------------|
| <b>Руководитель</b> | <u>                    </u><br>(дата) | <u>                    </u><br>(подпись) | <u>Марченко Е.В.</u><br>(Ф.И.О.) |
|---------------------|---------------------------------------|------------------------------------------|----------------------------------|

Санкт-Петербург  
2025 г.

## **Цель**

Овладеть практическими навыками и умениями реализации web-сервисов и использования сокетов.

## **Задачи**

1. Реализация простейшей клиент-серверной логики типа «Hello, world»;
2. Реализация клиент-серверной логики для подсчёта площади трапеции;
3. Реализация клиент-серверной логики для отображения html-страницы;
4. Реализация многопользовательского сайта.

## Ход работы

### Задание 1

#### *Реализация простейшей клиент-серверной логики типа «Hello, world»*

В первую очередь, импортируем библиотеку `sockets`. Далее создаем наш сервер. Привязываем его к порту 9090, после чего выставляем возможность слушать лишь одного клиента.

При подключении клиента выводим в консоль сообщение об успешном подключении и в цикле принимаем ответы с помощью `recv`. Проверяем равенство сообщения «Hello, server». Если равенство верно, то отправляем ответ «Hello, client», иначе «Error».

```
import socket

sock = socket.socket()
sock.bind(('', 9090))
sock.listen(1)
conn, addr = sock.accept()

print('connected:', addr)

while True:
    data = conn.recv(2048)
    if not data:
        break
    print(data)
    if data == b'Hello, server!':
        conn.send(b'Hello, client!')
    else:
        conn.send(b'Error')

conn.close()
```

Рисунок 1 – Код для создания сервера в Задании 1

Далее необходимо создать код клиента. В нем, в первую очередь, создается сокет, который подключается к порту 9090. После чего отправляется сообщение «Hello, server». Далее идет прием ответов, после чего осуществляется вывод в консоль.

```
import socket

sock = socket.socket()
sock.connect(('localhost', 9090))
sock.send(b'Hello, server!')

data = sock.recv(1024)
sock.close()

print(data)
```

Рисунок 2 – Код для создания клиента в Задании 1

```
C:\Users\aleks\AppData\Local\Progr  
b'Hello, client!'  
  
Process finished with exit code 0
```

Рисунок 3 – Вывод в консоли клиента

## Задание 2

### *Реализация клиент-серверной логики для подсчёта площади трапеции*

Данное задание делается по аналогии с предыдущим, только вместо текста передаются числа: две стороны и высота.

Код для клиента также создает сокет, далее пользователь вводит с клавиатуры 3 числа. Передаются они с помощью json-a, в который упаковываются числа. После чего ожидает ответ от сервера в формате json, который потом декодируется и выводится пользователю.

```
import socket
import json

sock = socket.socket()
sock.connect(('localhost', 9090))
a, b = map(int, input("Введите верхнюю и нижнюю сторону трапеции через пробел: ").split(' '))
h = int(input("Введите высоту трапеции: "))
sock.send(json.dumps([a, b, h]).encode())

data = json.loads(sock.recv(1024).decode())
sock.close()

print("Площадь трапеции равна", data[0])
```

Рисунок 4 – Код клиента для Задания 2

Код для клиента отличается от предыдущего задания лишь приемом json-a от клиента, его декодированием, вычислением площади и передачей его с помощью json-a обратно клиенту.

```
import socket
import json

sock = socket.socket()
sock.bind(('', 9090))
sock.listen(1)
conn, addr = sock.accept()

print('connected:', addr)

while True:
    data = conn.recv(2048)
    if not data:
        break
    data = json.loads(data.decode())
    square = ((data[0] + data[1]) / 2) * data[2]
    conn.send(json.dumps([square]).encode())

conn.close()
```

Рисунок 5 – Код для сервера в Задании 2

```
C:\Users\aleks\AppData\Local\Programs\Python\Python39\python
Введите верхнюю и нижнюю сторону трапеции через пробел: 4 6
Введите высоту трапеции: 3
Площадь трапеции равна 15.0

Process finished with exit code 0
```

Рисунок 6 – Вывод в консоли клиента

### Задание 3

#### *Реализация клиент-серверной логики для отображения html-страницы*

В данном задании необходимо было, чтобы сервер на ответ клиента передавал клиенту html-страницу и открывал ее в браузере.

В коде клиента много отличий, по сравнению с прошлым заданием, хотя структура остается такой же. На сервер посылается ответ в формате http-запроса, взятого из файла с заданием. После чего ожидается ответ. После его получения, происходит запись ответа в html-файл. И далее с помощью библиотеки webbrowser происходит открытие данного файла в браузере.

```
import socket
import json
import webbrowser
import os

sock = socket.socket()
sock.connect(('localhost', 9090))
sock.send(b"GET / HTTP/1.1\r\nHost: localhost\r\n\r\n")

data = sock.recv(4096).decode('utf-8')

with open('index_new.html', 'w', encoding='utf-8') as file:
    file.write(data)

webbrowser.open('file://' + os.path.realpath('index_new.html'))

sock.close()
```

Рисунок 7 – Код клиента в Задании 3

В коде сервера также произошли некоторые изменения в обработке ответа. После его получения сервер записывает в переменную содержимое приготовленного заранее файла index.html. Далее отправляет его в формате, представленном в файле с заданием, клиенту.

```

sock = socket.socket()
sock.bind(('localhost', 9090))
sock.listen(1)
conn, addr = sock.accept()

print('connected:', addr)

while True:
    with open('index.html', 'r', encoding='utf-8') as file:
        html_data = file.read()

    data = conn.recv(2048)
    if not data:
        break

    response = (
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: text/html; charset=utf-8\r\n"
        f"Content-Length: {len(html_data.encode('utf-8'))}\r\n"
        "Connection: close\r\n\r\n"
        f"{html_data}"
    )
    conn.send(response.encode('utf-8'))

```

Рисунок 8 – Код сервера в Задании 3

← ↻ 📄 file:///C:/Users/aleks/OneDrive/Рабочий%20стол/итмо/5%20сем/Web

HTTP/1.1 200 OK Content-Type: text/html; charset=utf-8 Content-Length: 386 Connection: close

## Сервер

Поздравляю вас, вы очутились на сервере Алексева Тимофея!

Рисунок 9 – Ответ сервера клиенту в Задании 3



## Задание 4

### *Реализация многопользовательского сайта*

В данном задании необходимо было создать многопользовательский чат, то есть отображение сообщений у всех клиентов после их отправления.

В коде клиента для начала создавался сокет, привязывался к порту, после чего у пользователя запрашивали его имя в чате. Далее создавались два потока, который отвечает за прием и отправку сообщений.

```
sock = socket.socket()
sock.connect(('localhost', 9090))

name = input('Введите ваше имя: ')

receive_thread = threading.Thread(target=receive_messages, args=(sock,))
receive_thread.start()

send_thread = threading.Thread(target=send_messages, args=(sock,))
send_thread.start()
```

Рисунок 10 – Создание клиентов в Задании 4

В обеих функциях запускаются бесконечные циклы. В одном работает функция, отвечающая за прием сообщений. Она декодирует сообщение. Если оно равно «name», значит, это рассылка сервера, которая необходима для утверждения имени пользователя в системе. В ином случае сообщения печатается пользователю в консоль.

```
def receive_messages(conn):
    while True:
        try:
            message = conn.recv(1024).decode('utf-8')

            if message == "name":
                conn.send(name.encode('utf-8'))
            else:
                print(message, '\n')
        except Exception as e:
            print("Ошибка:", e)
            conn.close()
            break
```

Рисунок 11 – Функция получения ответов клиентом

В функции отправки сообщений происходит проверка на равенство введенного сообщения «exit». Если это верно, то пользователь отключается от сервера. В ином случае, введенное сообщение отправляется на сервер для пересылки другим пользователям.

```
def send_messages(conn):  
    while True:  
        message = input()  
        if message.lower() == 'exit':  
            conn.close()  
            break  
        print(f"{name}: {message}")  
        conn.send(message.encode('utf-8'))
```

Рисунок 12 – Функция отправки сообщений клиентом

В коде сервера для начала создается сокет, после чего создается список всех клиентов. После каждого подключения в цикле клиенту отправляется системное сообщение «name», после чего приходит ответ с именем клиента. Далее данные соединения и имени записываются в массив. После чего всем клиентам отсылается сообщение о входе в чат нового пользователя. После чего создается поток, отслеживающий сообщения пользователя.

```
clients = []  
  
sock = socket.socket()  
sock.bind(('localhost', 9090))  
sock.listen()  
  
while True:  
    conn, addr = sock.accept()  
  
    conn.send("name".encode('utf-8'))  
    client_name = conn.recv(1024).decode('utf-8')  
  
    clients.append((conn, client_name))  
  
    send_all_clients(f"{client_name} вошёл в чат".encode('utf-8'), conn)  
    # conn.send("Вы подключились к серверу".encode('utf-8'))  
  
    thread = threading.Thread(target=handle_client, args=(conn,))  
    thread.start()  
  
conn.close()
```

Рисунок 13 – Создание сервера и добавление клиентов

Функция, принимающая запросы клиентов, работает в потоке. Она проверяет, что сообщение не является пустым. Если оно не является таковым, то рассылает его всем клиентам. Иначе удаляет пользователя. То же происходит и при появлении ошибок.

```
def handle_client(conn):
    while True:
        try:
            message = conn.recv(1024)
            print(message)
            if message:
                current_name = find_name(conn)
                send_all_clients(f"{current_name}: {message.decode('utf-8')}".encode('utf-8'), conn)
            else:
                delete_client((conn, find_name(conn)))
                break
        except Exception as e:
            print('Ошибка:', e)
            delete_client((conn, find_name(conn)))
            break
```

Рисунок 14 – Функция обработки запросов

Функция отправки сообщения всем клиентам работает просто через цикл for. Внутри него есть проверка на совпадение получателя и отправителя, если она пройдена, и они не совпадают, что клиенту отправляется ответ. При любой ошибке клиент удаляется из чата.

```
def send_all_clients(message, sender):
    for client in clients:
        if client[0] != sender:
            print(client)
            try:
                client[0].send(message)
            except Exception:
                delete_client(client)
```

Рисунок 15 – Функция отправки сообщений всем клиентам

Функция удаления клиента из системы проверяет его наличие. После чего отправляет всем клиентам сообщение о его выходе из чата. Удаляет его из списка всех клиентов и закрывает соединение.

```
def delete_client(client):
    if client in clients:
        name = client[1]
        send_all_clients(f"{name} вышел из чата".encode('utf-8'), client[0])
        clients.remove(client)
        client[0].close()
```

Рисунок 16 – Функция удаления клиентов

Далее создадим 3 файла с разными названиями, но одинаковыми кодами клиента и протестируем работу чата.

```
C:\Users\aleks\AppData\Local\Programs\Python\Py
Введите ваше имя: Тимофей
Катя вошёл в чат

Дима вошёл в чат

Дима: Ребята, я люблю WEB!

Катя: И я, я тоже люблю этот предмет!

А я вообще сдал все лабораторные уже
Тимофей: А я вообще сдал все лабораторные уже
```

Рисунок 17 – Работа чата у клиента номер 1

## Вывод

В ходе выполнения лабораторной работы цель была достигнута. Овладели практическими навыками и умениями реализации web-сервисов и использования сокетов.

В течение выполнения работы были реализованы клиент-серверная логики типа «Hello, world», для подсчёта площади трапеции, для отображения html-страницы и реализации многопользовательского сайта.

Код можно найти в репозитории:  
[https://github.com/NorthPole0499/WebDevelopment\\_2024-2025/tree/lab\\_6](https://github.com/NorthPole0499/WebDevelopment_2024-2025/tree/lab_6)